

JACOB COLES · DATA SCIENTIST

Python for Business with Copilot

Day 2 · External data, planning and a project you can explain

Today: one clear task at a time

Start	Session	Question
09:00	D2-S01	Why is this December figure wrong?
09:25	D2-S02	Euro-area inflation from the OECD
10:45	D2-S03	When today's refresh fails
11:30	D2-S04	A chart the board can read
13:15	D2-S05	Planning scenarios for the CFO
15:00	D2-S06	Python in a script, Power BI and KNIME
15:45	D2-S07	A new sample, then your project

Breaks: 10:30 and 14:45 · Lunch: 12:15–13:15

D2-S01 · 09:00

Why is this December figure wrong?

Check
S01

Retrieve
S02

Refresh
S03

Visualise
S04

Plan
S05

Integrate
S06

Project
S07

A figure is a claim to check

Northbridge · December 2025 · N02

A colleague was asked for N02's December operating result before the pack was ready and produced it in a spreadsheet: **EUR 21,754.26**.

The spreadsheet looked up each account in the mapping and **left out rows it could not find**.

Can the manager use it?

Step 1: what is the independent figure?

case_pack/data/faulty/F04_unmapped_account/source_controls.csv

period	entity	expected_posted_amount
2025-12	N02	?

The unit sends its own total separately from the export. That is what makes it a check.

Step 2: find the rows behind the gap

EXERCISE

1. Open `transactions/transactions_2025-12_N02.csv` from the VS Code Explorer (not by double-clicking: Excel strips leading zeros)
2. Compare its account codes with `account_mapping.csv`
3. **Ctrl+F** the code that is missing, add the amounts, and compare with the gap

Step 3: join, control, message

The join

"Not found" was treated as "leave out". No error, no warning, a plausible number.

The control

C05 unmapped accounts fails and blocks release.
The unit's own total shows the gap at once.

KEY IDEA

The right message: "N02's December figure is missing EUR 1,690.57 of posted revenue because account 0499 has no classification. Finance must classify it before the figure is used."

Two ways to run Python with Excel

	Python in Excel	Script writing a workbook
Where Python runs	Microsoft's cloud, from a <code>=PY(</code> cell	Your computer, from VS Code
What it reads	Data inside the workbook	Files in the course folder
Good for	Analysis where the data and audience stay in Excel	Several files, controls, a formatted pack every month

Success with one says nothing about the other.

Next: external data also needs a meaning, a source and a check.

D2-S02 · 09:25

Euro-area inflation from the OECD

Check
S01

Retrieve
S02

Refresh
S03

Visualise
S04

Plan
S05

Integrate
S06

Project
S07

A number from outside the company

Check
S01

Retrieve
S02

Refresh
S03

Visualise
S04

Plan
S05

Integrate
S06

Project
S07

The manager asks: **was euro-area inflation higher or lower than Sweden's in December 2025, was that typical, and how current is the figure?**

Today: ask the OECD for a small piece of data, **look at what comes back**, turn it into a table, answer, and save it with its source.

WATCH OUT

Inflation is context. It does not explain Northbridge's costs and it is not a forecast input.

What an API is

Your request	The OECD's response
Endpoint: the dataset's web address	Status code: 200 means OK
Parameters: your choices, such as months	Content type: the format that came back
Header: the format you would like	Body: the data, as structured text

The OECD answers in **SDMX-JSON**: JSON (named sections in curly brackets) with a fixed layout for statistics. No password or key is needed.

Know what the number means first

Field	Meaning
value 2.1	Prices 2.1% higher than the same month a year earlier
unit_label	Percent per annum
period	The month measured
retrieved_at_utc	When it was downloaded, not the month it describes

HICP: harmonised consumer prices, compiled the same way across the EU so places can be compared.

Demonstration: the request

```
Endpoint:  https://sdmx.oecd.org/public/rest/data/
           OECD.SDD.TPS,DSD_PRICES_COICOP2018@DF_PRICES_C2018_ALL,1.0/
           SWE+EA.M.HICP.CPI.PA._T.N.GY
Parameters: startPeriod = 2025-09
           endPeriod = 2026-01
           dimensionAtObservation = AllDimensions
Header:    application/vnd.sdmx.data+json; version=2
```

(The endpoint is one address; it is split here to fit the slide.) The key `SWE+EA.M.HICP.CPI.PA._T.N.GY` reads: Sweden or euro area · monthly · HICP · consumer price index · percent · all items · not adjusted · growth over one year.

Look before you convert

"0:0:0:0:0:0:0:0:0:0": [2.2, 0, 0]

Position	Dimension	Value
0	REF_AREA	EA (Euro area)
1 to 7	the measure	M, HICP, CPI, PA, _T, N, GY
8	TIME_PERIOD	2025-04

structures holds the **meaning**; dataSets holds the **numbers**, stored under positions.

Your turn (20 min)

EXERCISE

Notebook Steps 3 to 8

1. Choose the months around December 2025
2. Get the saved response; look at it; turn it into a table (three **OK** lines)
3. **YOUR TURN:** Copilot prompt in Step 7 → gaps : one row per month, SWE, EA, gap = EA – SWE
4. Save the table with its source record

KEY IDEA

The gap between two percentages is in **percentage points**: 2.1% to 2.0% is –0.1 percentage points.

The source record

outputs/D2-S02/source_record.json

```
"data_status": "cached",  
"retrieved_at_utc": "2026-09-13T18:33:16Z",  
"use": "External context only. Not a Northbridge forecast input."
```

A saved response keeps its **original** retrieval time. Analysing it today does not make it new.

Answer the manager

Month	Sweden %	Euro area %	Gap, points
2025-11	2.2	2.1	−0.1
2025-12	2.1	2.0	−0.1
2026-01	2.0	1.7	−0.3

Euro-area inflation was slightly **lower** in December. Prices still **rose**, just more slowly.

Next: what happens when tomorrow's request fails?

D2-S03 · 10:45

When today's refresh fails

Check
S01

Retrieve
S02

Refresh
S03

Visualise
S04

Plan
S05

Integrate
S06

Project
S07

When today's request fails



A refresh reruns the request from D2-S02. Sometimes it fails.

A **saved copy** (cache) may still be useful, but only if it keeps its **original download date** and is not too old.

Today: write the one rule that decides “fresh enough?”, then watch it work when requests fail.

Two facts, never merged

Field	Question	Values
refresh_outcome	What happened when we tried?	success, http_error, timeout, ...
data_status	How fresh is what we hand over?	live, cached, stale, unavailable

PREDICT FIRST

Today's request fails in every case. Label each: **A** copy 3 days old · **B** 30 days · **C** 36 days · **D** no copy

Where your rule fits

```
try the request —success→ save new copy → live
    |
    failure
    |
saved copy? —no→ unavailable
    |yes
YOUR RULE: age <= limit? → cached / stale
```

The original `retrieved_at_utc` never changes on failure; only `last_attempt_utc` does.

Your turn: the rule (10 min)

EXERCISE

Step 2 · sessions/D2-S03/learner/freshness_rule.py

```
def data_status_for_saved_data(age_in_days, max_age_days):  
    # YOUR TURN: "cached" if age_in_days <= max_age_days, otherwise "stale"
```

Run the file → four lines end in ok

WATCH OUT

“At most 35 days” **includes** 35. Using `<` instead of `<=` fails the 35-day test.

Run the scenarios (Step 3)

Case	Saved copy age	Today	refresh_outcome	data_status	retrieved_at_utc
A	3 days	server error (HTTP 500)	http_error	cached	2026-09-13T18:33:16Z
B	30 days	server error (HTTP 500)	http_error	cached	2026-09-13T18:33:16Z
C	36 days	server error (HTTP 500)	http_error	stale	2026-09-13T18:33:16Z
D	no saved copy	no answer (timeout)	timeout	unavailable	none

A stricter policy (Step 4)

PREDICT FIRST

"What if we only accepted data up to 20 days old?" Which case changes?

`MAX_AGE_DAYS = 20` → only **B** becomes **stale**. Outcomes, dates and values stay the same.

Set it back to **35**: changing an exercise setting does not change the policy.

The operator note

- Case C for the manager: "Today's refresh failed. The latest data we hold was downloaded on 13 September and is older than our 35-day limit, so treat it as out of date until a refresh succeeds."
- A script you run by hand is **rerunnable**, not a **scheduled service**.
- Another source needs its own checks: meaning, access terms, format, units, limits, dates.

Next: a chart the board can read.

D2-S04 · 11:30

A chart the board can read

Check
S01

Retrieve
S02

Refresh
S03

Visualise
S04

Plan
S05

Integrate
S06

Project
S07

The manager has one question

Why is December's operating result EUR 269.14 below budget?

We can show the arithmetic contributions. We still need other evidence to explain the business causes.

Today: read five rows → run a three-bar chart → improve one setting → justify a tool choice.

Five rows, three contributions

Step	Type	EUR
Budget operating result	Total	67,600.00
Revenue vs budget	Change	-2,972.11
Direct cost vs budget	Change	+1,778.66
Overhead vs budget	Change	+924.31
Actual operating result	Total	67,330.86

Plot the three **changes**. Adding budget and actual to them has no useful meaning.

A positive cost variance is good news

actual – budget = favourable variance

Example	Calculation	Meaning
Revenue below budget	$90 - 100 = -10$	Unfavourable
Costs below budget	$-90 - (-100) = +10$	Favourable

The bars are **differences**, not revenue and spending totals.

What should a reader see immediately?

- Scope: December 2025, all three units, EUR.
- A zero line: favourable above; unfavourable below.
- Values and words: meaning survives a monochrome print.
- A factual title: what the numbers show, without guessing causes.

Predict: what will adding value labels make easier?

Choose the tool for the team

Need	Question to ask
One monthly chart in a workbook	Does the standard Excel chart already meet the need?
Repeated exports with precise annotations	Can the team maintain and rerun a Python script?
Shared interactive report	What access and refresh route will users actually have?

Excel offers column and waterfall charts. Complexity alone is not an advantage.

Which misunderstanding are you solving?

Choice	Reader benefit	Tradeoff
Value labels	Exact amounts and sign words	More text
Colour by sign	Faster scanning	Still needs non-colour meaning
Separate net reference	Shows overall variance	Must not look like another contribution

Improve it for the board member

EXERCISE

Instructions Steps 2 and 3 · 17 minutes · `sessions/D2-S04/learner/variance_chart.py`

1. Decide which misunderstanding matters: direction, size or the overall result.
2. Change the settings you think will help (True or False , no quotation marks). Save with **Ctrl+S**.
3. Run the file again (**Run Python File**), then close and reopen the PNG.
4. Ask Copilot for the one line that orders the bars by size; paste it under YOUR TURN and run again.
5. Check the three amounts against the table, and judge whether your choice helped.

A setting that makes no visible difference is a finding to explain, not a failure.

Check the bridge with a calculator

$$-2,972.11 + 1,778.66 + 924.31 = -269.14$$

$$67,600.00 - 269.14 = 67,330.86$$

Costs offset **EUR 2,702.97** of the revenue shortfall.

A net bar is a summary of the three contributions. Never count it a second time.

The conclusion needs a limit

"Across all three units in December 2025, operating result was EUR 269.14 below budget. Revenue was EUR 2,972.11 below budget; favourable cost variances offset EUR 2,702.97."

Still unknown: whether prices, volumes, timing or something else caused those differences.

Keep: PNG + checked CSV + your own conclusion and tool choice.

D2-S05 · 13:15

Planning scenarios for the CFO

Check
S01

Retrieve
S02

Refresh
S03

Visualise
S04

Plan
S05

Integrate
S06

Project
S07

A useful planning answer has three parts

What might January–March 2026 look like for Northbridge?

1. A baseline we can explain.
2. Assumptions we can change.
3. Checks and limitations we can show.

Today you check a supplied baseline, choose two cases, check one month by hand and recommend.

Three tables, three jobs

Table name	What one row means	Size
history	One observed month / unit / report line	324
baseline	One forecast month / unit / report line	27
assumptions	One set of three multipliers	3

History: **January 2023–December 2025**. Forecast: **January–March 2026**.

The cutoff is a knowledge boundary

Latest allowed observation: December 2025.

- History must have all 36 months for each unit and line.
- January reporting actuals were not known at that cutoff.
- OECD observations remain external context, outside this model.
- Look across years for repeated patterns and changing levels.

A missing month is a data gap, not zero activity.

Seasonal naive: the same month last year

Forecast row	Historical source	Amount EUR
N02 direct cost, February 2026	February 2025	-19,648.94

Rule: copy the same calendar month one year earlier.

```
source_period = 2025-02 · data_cutoff = 2025-12
```

Easy to explain; it can miss a rising or falling trend.

Test on a past future

Pretend we are at **September 2025**:

Forecast October–December 2025 from October–December 2024 → compare with actuals later.

Held-out line	Mean absolute error, EUR	Observations
Revenue	2,747.84	9
Direct cost	1,241.81	9
Overhead	183.55	9

MAE: average the **size** of the errors. It is not a probability.

Assumptions multiply baseline levels

Scenario	Revenue	Direct cost	Overhead
baseline	×1.00	×1.00	×1.00
growth	×1.10	×1.08	×1.00
cost_pressure	×1.00	×1.12	×1.05

A 1.12 multiplier means **12% above the baseline level**. Each month’s baseline is multiplied once; no monthly compounding.

Trace one negative cost through the calculator

$$-19,648.94 \times 1.12 = -22,006.8128$$

Round the line to cents → **-22,006.81 EUR**.

More negative cost → lower operating result, other lines unchanged.

The supplied function chooses the matching multiplier, multiplies once and rounds to cents.

The calculation is supplied; the choice is yours

Notebook Step 4: **81 original scenario rows** · neutral baseline unchanged.

Finance expects supplier costs about **12% above baseline** and sales about **5% above baseline**.

Translate those expectations into factors. Predict the result before running.

Give the CFO a useful comparison

EXERCISE

Notebook Step 5 · edit the four numbers, then run the cell

Case	Your decision
Finance case	Translate “sales about 5% above baseline” and “direct costs about 12% above baseline” into two multipliers
Your case	Between finance’s case and the original growth plan (revenue 1.10, direct cost 1.08), with a business reason

Overhead stays at 1.00 in both. Write multipliers as numbers, for example `1.05`, with no percent sign. The checks accept factors from **0.5 to 1.5**; they cannot justify an assumption for you.

Check one month by hand

N02, February 2026

Line	Baseline EUR	Original growth	Finance case
Revenue	52,688.88	×1.10	×1.05
Direct cost	−19,648.94	×1.08	×1.12
Overhead	−7,650.54	×1.00	×1.00

Round each line to cents, then add the three signed amounts.

Separate observations and assumptions

- History: observed monthly amounts, up to December 2025.
- Future: original growth, finance case and your chosen case.
- Same scope: all three units, EUR operating result.
- Visible cutoff, readable legend and distinct markers.
- Overlapping lines may be a valid result; inspect the assumptions.

The gap between scenarios is not a confidence interval. Compare it with the past errors from Step 3.

Recommend what the CFO should do next

Notebook Step 8 · `interpretation.md`: scope · the two cases and why · prediction and hand check · biggest driver · recommendation · limits

Hand in **`changed_assumptions.csv`**, **`scenario_figure.png`**, **`interpretation.md`**.

Restart the kernel and rerun to check repeatability.

D2-S06 · 15:00

Python in a script, Power BI and KNIME

Check
S01

Retrieve
S02

Refresh
S03

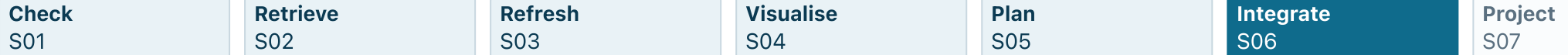
Visualise
S04

Plan
S05

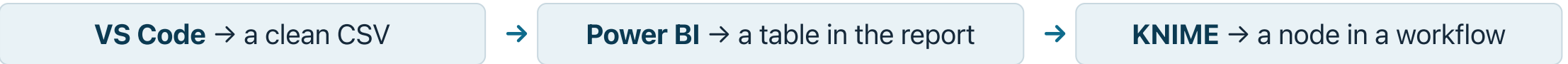
**Integrate
S06**

Project
S07

One Python step, three homes



The manager wants one operating result per unit. You will write that step **once** and run it in:



Then decide which route suits a colleague.

Step 1: write it in VS Code (8 min)

```
result = (dataset.groupby(["period", "entity"], as_index=False)["actual_amount"].sum()  
          .rename(columns={"actual_amount": "operating_result"}))
```

Unit	Operating result EUR
N01	29,862.50
N02	23,444.83
N03	14,023.53

KEY IDEA

Power BI and KNIME hand Python the table as **dataset** . Your step creates **result** .

Step 2: Power BI Desktop (12 min)

1. **Get data > Text/CSV** (full path) → **Transform Data**
2. `period` type → **Text**
3. **Transform > Run Python script** → paste your lines
4. Click **Table** next to `result` → **Close & Apply**
5. Table visual: `entity`, `operating_result`

WATCH OUT

Do not paste the lines that read the CSV. Power BI already supplies `dataset`.

Step 3: KNIME (12 min)

CSV Reader → Python Script → CSV Writer

```
import knime.scripting.io as knio
import pandas as pd
dataset = knio.input_tables[0].to_pandas()
# your lines creating result
knio.output_tables[0] = knio.Table.from_pandas(result)
```

First and last lines connect Python to the workflow. The middle is **the same step**.

Where does Python run, and who keeps it working?

	Clean CSV	Power BI step	KNIME node
Python runs	Script author's computer	Inside Power BI Desktop	Inside KNIME
Colleague needs to use it	Excel	Power BI + the report	KNIME + the workflow
Colleague needs to update it	Python + script	Power BI and its Python	KNIME + Python Integration

A Python **visual** returns a picture, not a table.

Step 4: decide the route

EXERCISE

outputs/D2-S06/route_decision.md

- Did all three results match?
- Where did Python run in each?
- Which colleague, what do they do monthly?
- Which route, and who keeps it working?
- What have you not tested?

Next: choose your project for Days 3 and 4.

D2-S07 · 15:45

A new sample, then your project

Check
S01

Retrieve
S02

Refresh
S03

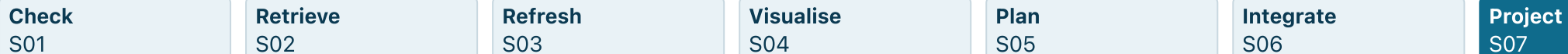
Visualise
S04

Plan
S05

Integrate
S06

Project
S07

Two days that are yours



Days 3 and 4: **your own project**, with the instructor's support.

Today, in 75 minutes:

1. **A new sample** you have not seen: what can you already do alone?
2. **Choose a track** and write a short plan.
3. **First run** of your starting point.

Part 1: February for N01 (20 min)

Rules	
Scope	N01, February 2026 (the files also contain N02)
Rows	posted, EUR
Signs	keep them: refunds and reversals
Amount	loaded as text : convert before adding
Mapping	account_code as text, left join, stop on duplicates

EXERCISE

Open `sessions/D2-S07/learner/my_analysis.py` , run it, predict with a calculator, ask Copilot, run again, compare.

Six tracks, each with a worked example

Track	Built on
Monthly reporting pack	D1-S06 to D1-S08
Audit exceptions	D1-S06
Multi-file reconciliation	D1-S04
Budget scenarios	D2-S05
External refresh with controls	D2-S02, D2-S03
Management dashboard (Excel or Power BI)	D2-S04, D2-S06

Each folder: a plain README and `worked_example.py` that runs end to end.

A plan small enough to finish

`outputs/projects/<name>/plan.md`

- The business question and who uses it
- The output and where it is saved
- The starting point and the **one change** you will make
- **One figure** you can check without trusting your code
- One faulty input and what should happen
- Day 3 and Day 4 steps, and what you leave out

EXERCISE

Then let Copilot attack it (5 min): *“Do not write code and do not tell me it looks good. Ask me the five questions most likely to show this plan is too large, that my check proves less than I think, or that I have assumed data I may not have. Then name the one part you would cut.”*

Write the two questions you cannot answer into the plan, and bring them to the review.

Every finished project shows

- **Why this tool** was the simplest sustainable choice
- **Evidence of checks:** at least one independent figure and one failure case
- **A handover note:** purpose, inputs, how to run, what was checked, limits

Day 4 ends with short presentations if time allows.

First run (15 min)

EXERCISE

1. Copy your track's `worked_example.py` into your project folder and run **the copy**: the same output means your starting point works
2. Make the first small change from your plan, and run it again
3. Note tomorrow's first step in `plan.md`
4. Start `worklog.md`: **what I asked for** · **what I kept** · **what I checked**, one line per step